

AERO70011 HPC Coursework Report: Poisson Problem Solver

Haobo Du

CID: 02372582

Abstract

This report presents the development and optimisation of a parallel Poisson equation solver using MPI and GPU acceleration. A 3D Jacobi iteration with Cartesian domain decomposition is implemented, with halo exchange used for inter-process communication. Performance is improved by reducing the frequency of global residual evaluations. Both strong and weak scaling tests are conducted and evaluated. GPU acceleration is applied to the Jacobi iteration, residual computation, and a part of the halo exchange. The final implementation achieves a speedup of about $21\times$ compared to the optimised MPI solver while maintaining convergence behaviour.

03/19/2026

List of Figures

1	Two-dimensional index plot.	3
2	Flattened one-dimensional index plot.	3
3	MPI 3D Domain Decomposition.	4
4	Illustration of Halo Exchange in 2D (x-direction).	4
5	Comparison between plain and optimised MPI with grid size of 96^3	5
6	Strong scaling speed-up plots for optimised MPI	5
7	Runtime for weak scaling with fixed local grid of 32^3	6
8	Weak scaling efficiency with varying core number	6

List of Tables

1	Performance Analysis and Time Distribution (Grid Size: 96^3)	5
2	Strong Scaling Performance Analysis (Fixed Global Grid Size: 96^3)	5
3	Weak Scaling Performance Analysis (Fixed Local Grid Size: 32^3)	6
4	Performance comparison between optimised MPI and CUDA implementations for 96^3 grid (1 core)	6

1 MPI Parallelisation Approach

1.1 Index Flattening

Prior to implementing the MPI, the 3D grid is linearized into a 1D array to enable efficient data processing and memory management. This mapping transforms 3D coordinates (i, j, k) into a linear index using the row-major convention:

$$\text{index}(i, j, k) = i \cdot N_y N_z + j \cdot N_z + k \quad (1)$$

This approach simplifies the extraction of data sub-domains for later calculation and parallelisation. As illustrated in the transition from the 2D visualization (Figure 1) to the flattened 1D representation (Figure 2), this indexing scheme provides a consistent framework for traversing the global data structure.

Each row is traversed with k (fastest direction)

j = 0	(2,0,0) 0	(2,0,1) 1	(2,0,2) 2	(2,0,3) 3	(2,0,4) 4
j = 1	(2,1,0) 5	(2,1,1) 6	(2,1,2) 7	(2,1,3) 8	(2,1,4) 9
j = 2	(2,2,0) 10	(2,2,1) 11	(2,2,2) 12	(2,2,3) 13	(2,2,4) 14
j = 3	(2,3,0) 15	(2,3,1) 16	(2,3,2) 17	(2,3,3) 18	(2,3,4) 19

Figure 1: Two-dimensional index plot.

j = 0 block					j = 1 block					j = 2 block					j = 3 block				
0 (0,0)	1 (0,1)	2 (0,2)	3 (0,3)	4 (0,4)	5 (1,0)	6 (1,1)	7 (1,2)	8 (1,3)	9 (1,4)	10 (2,0)	11 (2,1)	12 (2,2)	13 (2,3)	14 (2,4)	15 (3,0)	16 (3,1)	17 (3,2)	18 (3,3)	19 (3,4)

Figure 2: Flattened one-dimensional index plot.

1.2 Decomposition and Communication

The 3D Poisson solver is parallelised using MPI with a 3D Cartesian domain decomposition inspired by the 2D approach presented by Mitchell et al [1]. The global domain of size $N_x \times N_y \times N_z$ is split across a process grid of size $P_x \times P_y \times P_z$, and each MPI rank owns one rectangular subdomain as illustrated by Figure 3. With determined dimensions, the Cartesian communicator is then constructed with `MPI_Cart_create()` and `MPI_Cart_coords()`.

The Jacobi iteration is used for solving the Poisson equation, where the iteration itself requires a 7-point stencil (the update point itself and its six neighbours). To implement this in a parallel environment in a more efficient manner, "halo layers" are integrated into each local subdomain to store boundary data from adjacent processes. Consequently, the `local_index()` inline function incorporates a +2 to extend local array dimensions by two cells in each direction to accommodate one halo layer on either side.

For grid points located on the boundary of a local subdomain, one or more of these neighbours belong to an adjacent MPI process rather than the locally owned region. To maintain mathematical correctness, these boundary values must be shared across the process through exchange between subdomains. While the exchange is necessary, the way data is stored in memory is not ideal for this process.

Since each face is not stored as one contiguous memory block in the flattened 1D layout, the boundary planes are first packed into contiguous communication buffers using `pack_face_x/y/z()` to rearrange for MPI communication. The actual data exchange is then performed by `MPI_Sendrecv()`, which simultaneously transmits the packed buffer to a neighbor and receives the corresponding boundary data into a local `recvbuf`. Finally, `unpack_face_x/y/z()` functions are used to map these received values back into the offset halo layers of the local subdomain. This is integrated as a single `swap_space()` function in the code. The exchanging process is demonstrated by Figure 4, where the point lies on the boundary and requires point at $i = 0$ in the halo layer, thus the required values are obtained from the neighbouring process and stored in corresponding halo layer.

Once the local Jacobi update and halo exchanges are complete, the solver must determine if the global solution has reached the specified tolerance (ϵ). Since each MPI process only computes the error for its own subdomain, a global consensus is required to terminate the iteration loop. This is done by `residual_global()` function in the code. Each process first calculates its local residual, which is then

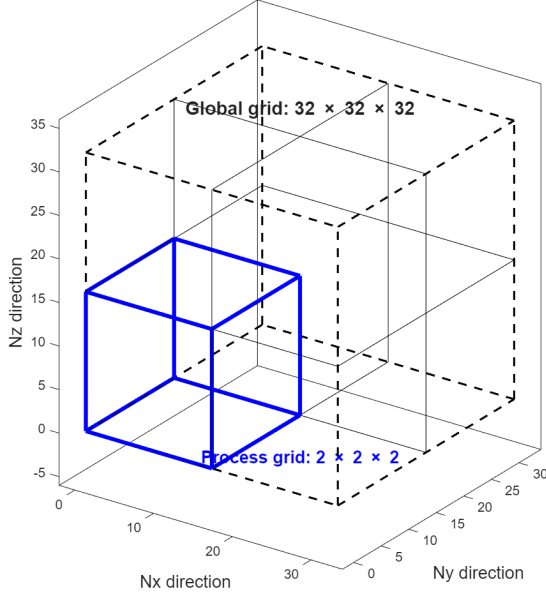


Figure 3: MPI 3D Domain Decomposition.

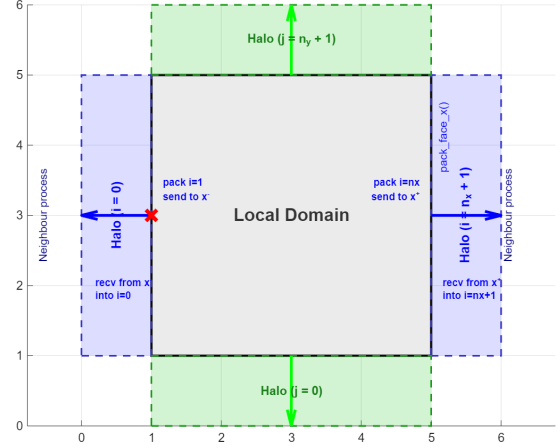


Figure 4: Illustration of Halo Exchange in 2D (x-direction).

aggregated across the entire communicator using `MPI_Allreduce()`. This operation ensures that every rank receives the same global L_2 norm and is able to evaluate the exit condition identically.

After convergence is reached, the distributed subdomains should be united by the root process (rank 0) and produce a single output file. To do this, the halo layers should be removed first to produce only interior solution. This is done by `pack_block()` in the code. The data blocks are then transmitted with `MPI.Send()` to the root. At root process, the function `unpack_block()` maps each received segment into a global array using each rank's starting coordinates (s_x, s_y, s_z). The final solution is saved using `write_solution()`.

This parallelisation strategy is efficient as it aligns closely with the 7-point stencil requirement for Jacobi implementation. A 3D Cartesian domain decomposition distributes the global grid across a process grid, ensuring balanced workload while restricting the communication to only six neighbouring processes. Communication overhead is minimised by exchanging only boundary planes via halo layers. The use of `swap_space()` isolates the exchanging process and enables a clear separation between communication and computation, so that `jacobi_local()` operates entirely on the locally available data. Although more advanced optimisation such as non-blocking communication could further improve the performance, the current approach provides an effective and well-matched parallelisation for given problem.

2 MPI Performance Optimisation

By performing verification test 2 with different core numbers on a fixed grid size, it is observed that the Jacobi iteration and residual evaluation dominate the overall runtime as shown by Table 1. However, Jacobi iteration is already a purely local operation and performs efficiently, while the residual evaluation involves global communication and `MPI_Allreduce()` is time-consuming. The `residual_global()` process should be the aim for optimisation.

The original residual evaluation method computed the residual at every iteration, requiring both a full local residual sweep and an `MPI_Allreduce()` operation. The optimisation implemented in `mpi_optimised.cpp` evaluates the residual only once every 20 iterations. This reduces the frequency of global synchronisation and significantly lowers the runtime, while maintaining similar convergence behaviour.

For validation, strong-scaling tests were performed using both original and optimised MPI and the results are compared in Figure 5.

Table 1: Performance Analysis and Time Distribution (Grid Size: 96^3)

Cores	Topology	Runtime (s)	Jacobi (%)	Halo Exchange (%)	Residual (%)
1	$1 \times 1 \times 1$	460.4	55.6%	0.002%	44.3%
2	$2 \times 1 \times 1$	233.9	54.8%	0.89%	44.3%
4	$2 \times 2 \times 1$	121.6	53.0%	2.20%	44.7%
8	$2 \times 2 \times 2$	68.1	47.1%	4.11%	48.7%

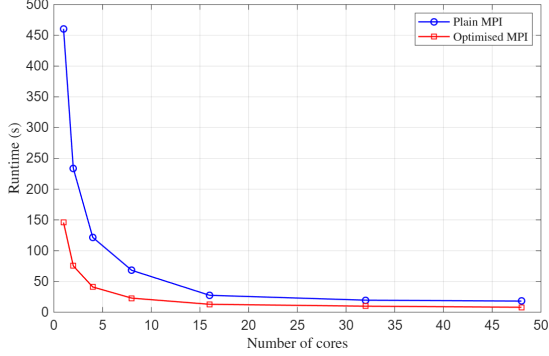
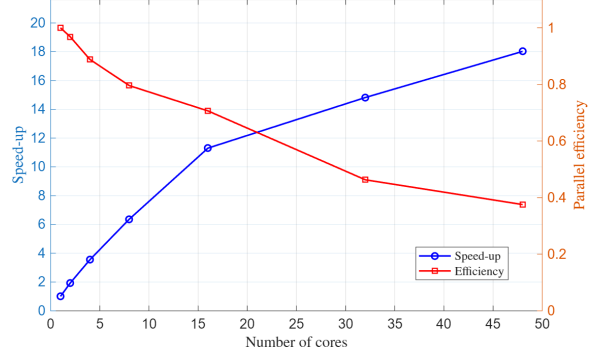
Figure 5: Comparison between plain and optimised MPI with grid size of 96^3 .

Figure 6: Strong scaling speed-up plots for optimised MPI

Table 2: Strong Scaling Performance Analysis (Fixed Global Grid Size: 96^3)

Cores	Runtime_s	Jacobi(s)	Halo Exchange (s)	Residual(s)	Speedup	Efficiency
1	145.8	139.6	0.0012	6.23	1.0000	1.00000
2	75.3	70.6	1.69	3.12	1.9363	0.96813
4	41.04	35.2	4.14	1.67	3.5526	0.88816
8	22.9	17.7	4.13	1.02	6.3668	0.79585
16	12.9	8.72	3.46	0.75	11.302	0.70640
32	9.84	6.02	3.07	0.74	14.817	0.46303
48	8.09	4.44	2.90	0.73	18.022	0.37546

The strong scaling results presented in Table 2, together with Figures 5 and 6, indicate that the optimised MPI solver effectively reduces the total runtime as the number of cores increases, demonstrating that the parallelisation strategy functions correctly. At lower core counts, the speedup increases nearly proportionally with the number of cores, and the efficiency remains relatively high. This suggests that the computational workload, dominated by the Jacobi iteration, is well distributed across MPI ranks.

However, at higher core counts, the scaling behaviour begins to deviate from the ideal case and the efficiency decreases. Although the total runtime continues to decrease, the rate of improvement becomes progressively smaller. This is because the global problem size is fixed, and each process is assigned a progressively smaller subdomain. Consequently, the amount of computation per core decreases rapidly, reducing the benefits gained from further parallelisation.

Moreover, as the number of cores increases further, the impact of communication becomes increasingly significant. The halo exchange time does not scale down at the same rate as the computation and begins to account for a larger proportion of the total runtime. This growing communication overhead, combined with increased synchronisation costs between processes, limits the achievable speedup and ultimately constrains the overall performance of the solver.

The weak scaling results are presented by Table 3, Figure 7 and 8. It reveals significant challenges in maintaining performance as the problem size grows. In an ideal weak scaling scenario, the runtime should remain rather constant since each process is assigned the same amount of work. However, the results show that the total runtime increases steadily with the number of cores, and results in a significant drop in efficiency. This indicates that the solver is unable to maintain performance under large system size.

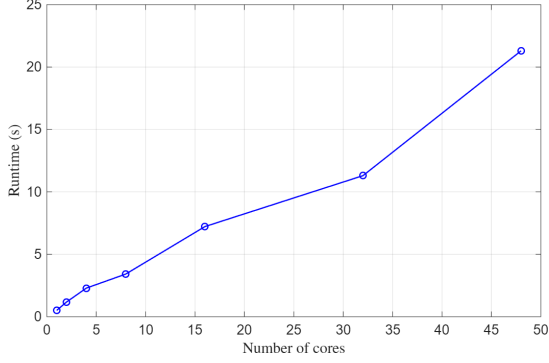


Figure 7: Runtime for weak scaling with fixed local grid of 32^3

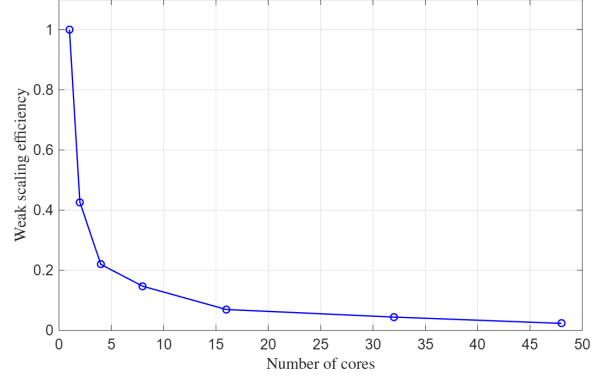


Figure 8: Weak scaling efficiency with varying core number

Table 3: Weak Scaling Performance Analysis (Fixed Local Grid Size: 32^3)

Cores	$P_x \times P_y \times P_z$	$N_x \times N_y \times N_z$	Runtime (s)	Iterations	Efficiency
1	$1 \times 1 \times 1$	$32 \times 32 \times 32$	0.503	5041	1.00000
2	$2 \times 1 \times 1$	$64 \times 32 \times 32$	1.18	10441	0.42627
4	$2 \times 2 \times 1$	$64 \times 64 \times 32$	2.29	16001	0.21965
8	$2 \times 2 \times 2$	$64 \times 64 \times 64$	3.42	21681	0.14708
16	$4 \times 2 \times 2$	$128 \times 64 \times 64$	7.22	44461	0.069668
32	$4 \times 4 \times 2$	$128 \times 128 \times 64$	11.3	68461	0.044513
48	$4 \times 4 \times 3$	$128 \times 128 \times 96$	21.3	77161	0.023615

The primary cause of this behaviour is the growth of communication overhead as more processes are introduced. Although the local grid size is fixed at 32^3 , the global domain expands with increasing core count, resulting in a larger number of subdomain interfaces. Consequently, the frequency and total amount of halo exchange increase, and more time is spent on inter-process communication. In addition, the number of iterations required also increases, further contributing to the increase in runtime. Furthermore, the increased number of MPI processes introduces additional synchronisation overhead.

These factors together prevent the solver from achieving ideal weak scaling. As a result, the current implementation becomes inefficient for large-scale problems.

3 GPU Results

Table 4: Performance comparison between optimised MPI and CUDA implementations for 96^3 grid (1 core)

Method	Runtime (s)	Jacobi (s)	Halo (s)	Residual (s)	Iterations
Optimised MPI	145.8	139.6	0.0012	6.23	50521
CUDA	6.93	6.56	0.0016	0.355	50521

The GPU offloading strategy focuses on parallelising the computationally intensive components of the solver, the Jacobi iteration and residual evaluation. The local 3D subdomain is mapped onto a 3D CUDA execution configuration using thread blocks of size $8 \times 8 \times 4$, enabling thousands of grid points to be updated concurrently and allowing the stencil computation to fully exploit GPU parallelism.

The same flattened data layout introduced in Section 1.1 is reused on the device, ensuring consistency between the CPU and GPU implementations while maintaining efficient memory access patterns. This avoids additional data restructuring and allows the stencil kernels to operate directly on the existing layout.

A buffering strategy similar to the MPI implementation is also applied, where separate arrays are used for the current and updated solutions. Instead of copying data between arrays, the device pointers are swapped after each iteration, eliminating unnecessary memory transfers and reducing overhead. In addition, part of the halo exchange process is also moved onto GPU. The boundary faces are packed and unpacked using CUDA kernels with updated packing and unpacking functions, `pack/unpack_face_x/y/z_kernel()`. The MPI communication itself is still performed through host memory.

Moreover, the global residual norm is computed using a hybrid GPU and MPI approach. Each MPI rank evaluates its local residual contributions on the GPU, and the resulting values are reduced on the device using the Thrust library [2], which provides optimised parallel reduction routines. The partial results are then combined across all processes using `MPI_Allreduce`. This avoids serial accumulation on the CPU and improves efficiency.

The CUDA implementation achieves a significant speedup compared to the optimised MPI version. Under the same configuration, the total runtime is reduced from 145.8 s (MPI) to 6.93 s (MPI+CUDA), corresponding to a speedup of approximately $21\times$. Both implementations converge in the same number of iterations (50,521), confirming that the numerical behaviour remains unchanged and that the performance improvement arises purely from accelerated computation.

During initial development, only the Jacobi iteration and residual evaluation were offloaded to the GPU using `jacob_kernel()` and `residual_kernel()`. However, this approach resulted in a runtime comparable to the MPI-only implementation, at approximately 146.9 s. Although the Jacobi iteration (8.78 s) and residual evaluation (0.67 s) were accelerated, halo communication dominated the runtime at 137.4 s.

This behaviour is due to the large amount of data transferred between host and device at each iteration, which significantly increased the communication overhead.

To address this issue, the previously mentioned CUDA-based packing and unpacking functions are thus implemented, so that only the required halo data is transferred during communication. This significantly reduces data movement and enables the GPU implementation to achieve substantial speedup.

References

- [1] Mitchell L, Schulz H, Arnold C. MPI: domain decomposition and halo exchanges. Durham University; 2022. Available from: <https://teaching.wence.uk/phys52015/exercises/mpi-stencil/>
- [2] NVIDIA Corporation. Thrust API documentation. CUDA C++ Core Compute Libraries (CCCL); 2026 [cited 2026 Mar 19]. Available from: <https://nvidia.github.io/cccl/unstable/thrust/index.html>